



TRADINGPRO

Kravspecifikation

Ozan Korkmaz

Vejleder: Flemming Sørensen

1205hf26086per | 12-08-2026

Indholdsfortegnelse

1. Indledning.....	2
1.1 Problemformulering.....	2
1.2 Formål.....	2
2. Læsevejledning	3
2.1 Aktør beskrivelse	3
2.1.1 Bruger (User)	3
2.1.2 Administrator (Admin).....	3
2.1.3 Superadministrator (SuperAdmin).....	3
3. Generel Systembeskrivelse	3
3.1 System beskrivelse	3
3.2 Hardware- og Softwarebeskrivelse.....	3
3.2.1 Backend dependencies.....	4
3.2.2 Frontend dependencies	4
3.3 Begrænsninger	5
3.4 Krav til udviklingsforløbet	5
3.4.1 Udviklingssprog og Frameworks.....	5
3.4.2 Udviklingsmiljøer og værktøjer	6
3.4.3 Versionskontrol	6
3.4.4 Dokumentation	6
3.4.5 Test af software	7
4. Specifikke Krav	7
4.1 Datakrav	7
4.1.1 Databaseformater	7
4.2 Funktionelle krav (Use Cases - Overblik).....	7
5. Eksterne Grænsefladekrav & Systemflow	7
5.1 UI/UX og Responsivitet.....	7
5.2 Systemflow (Arkitekturdiagram)	7
6. Tidsplan (Sprint-struktur over 4 uger).....	8
Bilag 1 – Use Cases	9

1. Indledning

Dette er en kravspecifikation omhandlende produktet TradingPro. TradingPro er en webbaseret platform, der udvikles som et projekt med fokus på overvågning og analyse af kryptovalutamarkedet.

Projektet har fokus på at udvikle en brugervenlig løsning, der samler realtidsdata, tekniske analyseværktøjer og kunstig intelligens på én platform. Formålet er at give brugeren bedre muligheder for at følge markedet og analysere dets udvikling.

Projektet definerer problemet, kravene til løsningen samt de funktioner og teknologier, der indgår i udviklingen af TradingPro.

1.1 Problemformulering

Moderne kryptohandlere og investorer har behov for hurtig og pålidelig adgang til markedetsdata for at kunne følge og analysere markedets udvikling. Eksisterende løsninger kan være begrænsede i forhold til visualisering af historiske tidsseriedata og integration af AI-baserede analyser. Samtidig stiller systemet krav til sikker håndtering af forskellige brugerroller og adgangsniveauer.

På baggrund af dette fokuserer projektet på følgende centrale spørgsmål:

- Hvordan kan der udvikles en højtydende arkitektur ved hjælp af .NET, React og TimescaleDB til effektiv håndtering og visualisering af finansielle tidsseriedata fra eksterne børser såsom Binance?
- Hvordan kan eksterne AI-API'er integreres for at understøtte automatiserede tekniske analyser af udvalgte kryptovalutaer?
- Hvordan kan der implementeres en sikker rollebaseret adgangskontrol (RBAC) med rollerne User, Admin og SuperAdmin?

1.2 Formål

Formålet med projektet er at udvikle en skalerbar og brugervenlig webapplikation til overvågning og analyse af kryptovalutamarkedet. Brugeren skal kunne følge live-priser gennem interaktive candlestick-grafer, vælge forskellige tidsrammer og få adgang til AI-baserede markedsanalyser.

Systemet skal desuden understøtte en sikker rollebaseret administration, hvor Admin og SuperAdmin kan administrere brugere og relevante systemfunktioner.

2. Læsevejledning

2.1 Aktør beskrivelse

Systemet opererer med tre distincte brugertyper (roller), som hver har definerede rettigheder:

2.1.1 Bruger (User)

En standardbruger har adgang til at oprette en profil, logge ind, søge efter handelssymboler (f.eks. BTCUSDT), se live og historiske grafer på tværs af tidsintervaller samt anmode om AI-baserede markedsanalyser for det valgte aktiv.

2.1.2 Administrator (Admin)

En administrator har alle standardbrugerens rettigheder samt udvidede rettigheder til systemmoderering. Dette inkluderer håndtering af indhold, overvågning af systemets API-integrationer samt administration af almindelige brugerkonti.

2.1.3 Superadministrator (SuperAdmin)

Superadministratoren har fuld, ubegrænset kontrol over hele systemet. Dette omfatter tildeling og ændring af administratorrettigheder, systemkonfiguration, databasevedligeholdelse samt overvågning af systemets globale ressourceforbrug og sikkerhedslogfiler.

3. Generel Systembeskrivelse

3.1 System beskrivelse

TradingPro tillader brugere at se live og historiske krypto-markedsdata på interaktive grafer samt modtage AI-baserede analyser. TradingPro er en webapplikation, hvor en bruger kan undersøge handelssymboler, følge levende prisopdateringer via realtids-streams og trække på kunstig intelligens til at vurdere markedet. Dertil kan administratorer styre systemet og brugerrettigheder.

3.2 Hardware- og Softwarebeskrivelse

Systemet er opbygget som en webbaseret løsning bestående af en frontend, en backend og en database. Arkitekturen er designet med fokus på skalerbarhed, sikkerhed og effektiv håndtering af realtidsdata.

- **Backend:** Udviklet i C# med ASP.NET Core Web API. Backend håndterer systemets forretningslogik, autentificering, autorisation, databasekommunikation samt kommunikation med eksterne tjenester. Realtidsdata videreformidles til frontend via SignalR.
- **Frontend:** Udviklet med React og TypeScript. Tailwind CSS anvendes til brugergrænsefladen, mens TradingView Lightweight Charts anvendes til visualisering af markeds- og candlestick-data.
- **Database:** Systemet anvender en kombination af PostgreSQL og TimescaleDB. PostgreSQL anvendes til relationelle data såsom brugere, roller og systemkonfiguration, mens TimescaleDB anvendes til tidsseriedata, herunder historiske og aktuelle kryptovalutapriser samt candlestick-data.
- **Realtidskommunikation:** WebSocket anvendes til modtagelse af realtidsdata fra eksterne tjenester såsom Binance, mens SignalR anvendes til videreformidling af relevante data fra backend til frontend.

3.2.1 Backend dependencies

- **ASP.NET Core:** Anvendes som grundlag for udvikling af backend-API'et og håndtering af server-side funktionalitet.
- **Entity Framework Core:** Anvendes som ORM til databasehåndtering og kommunikation med PostgreSQL.
- **JWT (JSON Web Token):** Anvendes til sikker autentificering og håndtering af brugerens session mellem klient og backend.
- **Microsoft.AspNetCore.Authentication.JwtBearer:** Anvendes til validering og håndtering af JWT-baseret autentificering.
- **WebSocket:** Anvendes til at modtage realtidsdata fra eksterne tjenester såsom Binance.
- **Microsoft.AspNetCore.SignalR:** Anvendes til realtidskommunikation mellem backend og klienten, blandt andet til live pris- og candlestick-opdateringer.
- **System.Text.Json:** Anvendes til serialisering og deserialisering af data fra eksterne API'er.
- **HttpClient:** Anvendes til kommunikation med eksterne tjenester og API'er, herunder Binance og AI-tjenester.

3.2.2 Frontend dependencies

- **React:** Anvendes som frontend-framework til opbygning af brugergrænsefladen og håndtering af applikationens komponenter.
- **TypeScript:** Anvendes til type-sikker udvikling og strukturering af frontend-koden.
- **Tailwind CSS:** Anvendes til opbygning af en responsiv og ensartet brugergrænseflade.
- **TradingView Lightweight Charts:** Anvendes til visualisering af finansielle data gennem interaktive candlestick- og prisgrafer.
- **SignalR Client:** Anvendes til at modtage realtidsdata fra backend og opdatere brugergrænsefladen uden behov for manuel genindlæsning.

3.3 Begrænsninger

Følgende funktioner er ikke inkluderet i projektets første version (V1.0):

- **Automatiseret handel:** Systemet understøtter ikke automatisk køb og salg direkte gennem eksterne børser. Funktionaliteten er afgrænset til markedssdata, overvågning og analyse.
- **AI-analyse:** AI-funktionaliteten er begrænset til tekstbaserede analyser og opsummeringer. Automatisk generering af tegninger eller markeringer direkte på graferne er ikke en del af V1.0.

3.4 Krav til udviklingsforløbet

Udviklingen af **TradingPro** gennemføres individuelt og struktureres som en agil og iterativ udviklingsproces. Systemets funktioner udvikles løbende, hvor de enkelte moduler implementeres, testes og tilpasses undervejs.

Git anvendes til versionsstyring af kildekoden og sikrer, at ændringer i projektet kan spores og håndteres struktureret.

Kravspecifikationen fungerer som det overordnede grundlag for udviklingen. Eventuelle ændringer eller afvigelser fra de oprindelige krav vurderes løbende og dokumenteres, hvis de har betydning for projektets funktionalitet eller omfang.

3.4.1 Udviklingssprog og Frameworks

C# er valgt som udviklingssprog til backend, da det giver god understøttelse af moderne webapplikationer og har stærk integration med **ASP.NET Core**. ASP.NET Core anvendes som framework til udvikling af backend og API, da det understøtter en struktureret arkitektur samt de nødvendige funktioner til sikkerhed og realtidskommunikation.

React og **TypeScript** anvendes til udviklingen af frontend. React gør det muligt at opbygge brugergrænsefladen som genanvendelige komponenter, mens TypeScript bidrager til en mere struktureret og typesikker kodebase.

Entity Framework Core anvendes til Object-Relational Mapping (ORM) mellem backend og databasen. Det gør det muligt at arbejde med databaseentiteter direkte gennem C#-modeller.

PostgreSQL anvendes til lagring af systemets relationelle data, såsom brugere, roller og systemkonfiguration. **TimescaleDB** anvendes som udvidelse til PostgreSQL til håndtering af tidsseriedata, herunder historiske og aktuelle markedsdata.

3.4.2 Udviklingsmiljøer og værktøjer

Visual Studio anvendes som udviklingsmiljøer til henholdsvis backend- og frontendudvikling.

pgAdmin anvendes til administration og overvågning af PostgreSQL-databasen samt til håndtering af databaseindhold.

PostgreSQL og **TimescaleDB** køres lokalt uden brug af containerisering eller Docker. Dette gør det muligt at udvikle og teste systemets databasefunktionalitet direkte i det lokale udviklingsmiljø.

Swagger / OpenAPI anvendes til dokumentation og test af backendens API-endpoints. Det giver mulighed for at afprøve API-kald og kontrollere request- og response-data direkte gennem Swagger-interfacet.

3.4.3 Versionskontrol

Git anvendes som versionskontrollsystem gennem hele udviklingsforløbet. Det bruges til at registrere og håndtere ændringer i kildekoden samt bevare tidligere versioner af projektet.

Da projektet udvikles individuelt, giver Git mulighed for at arbejde struktureret med ændringer og sikrer, at tidligere versioner kan genskabes ved behov.

3.4.4 Dokumentation

Der udarbejdes løbende dokumentation af systemets funktionalitet og tekniske opbygning. Dokumentationen vil blandt andet beskrive systemets arkitektur, centrale komponenter og relevante tekniske beslutninger.

Derudover vil systemets brugergrænseflade og funktionalitet blive dokumenteret gennem relevante diagrammer og use cases. Formålet er at skabe et tydeligt overblik over systemets opbygning og anvendelse.

3.4.5 Test af software

Da systemet udvikles individuelt, anvendes både automatiserede og manuelle tests gennem udviklingsforløbet.

- **Automatiserede tests:** Backendens centrale funktionalitet og API-endpoints testes med **xUnit** for at kontrollere, at forretningslogik og relevante funktioner fungerer som forventet.
- **Manuelle tests:** Frontendens brugergrænseflade, grafer, realtidsdata og AI-analyse testes løbende i browseren. Formålet er at sikre, at systemets funktioner fungerer korrekt, og at brugeroplevelsen er overskuelig og responsiv.

4. Specifikke Krav

4.1 Datakrav

4.1.1 Databaseformater

Systemet skal anvende **Entity Framework Core** med en *Code First*-tilgang, hvor databasen opbygges ud fra de modeller og konfigurationer, der defineres i backend-koden.

Fluent API og relevante Data Annotations anvendes til at definere tabeller, datatyper og relationer. Databaseændringer håndteres gennem Entity Framework Core migrations.

PostgreSQL anvendes til relationelle data, mens **TimescaleDB** anvendes til tidsseriedata. Tidsserietabeller konfigureres som *Hypertables* for effektiv håndtering af markedssdata.

4.2 Funktionelle krav

4.2.1 Use Cases

Se bilag 1 – Use Cases

5. Eksterne Grænsefladekrav & Systemflow

TradingPro udvikles som en webbaseret applikation, hvor brugerfladen skal gøre det muligt at interagere med systemets funktioner på en enkel og overskuelig måde. Systemet skal kunne håndtere visning af markedsddata, interaktive grafer og AI-baserede analyser samt kommunikere med backendens API'er og eksterne tjenester.

5.1 UI/UX og Responsivitet

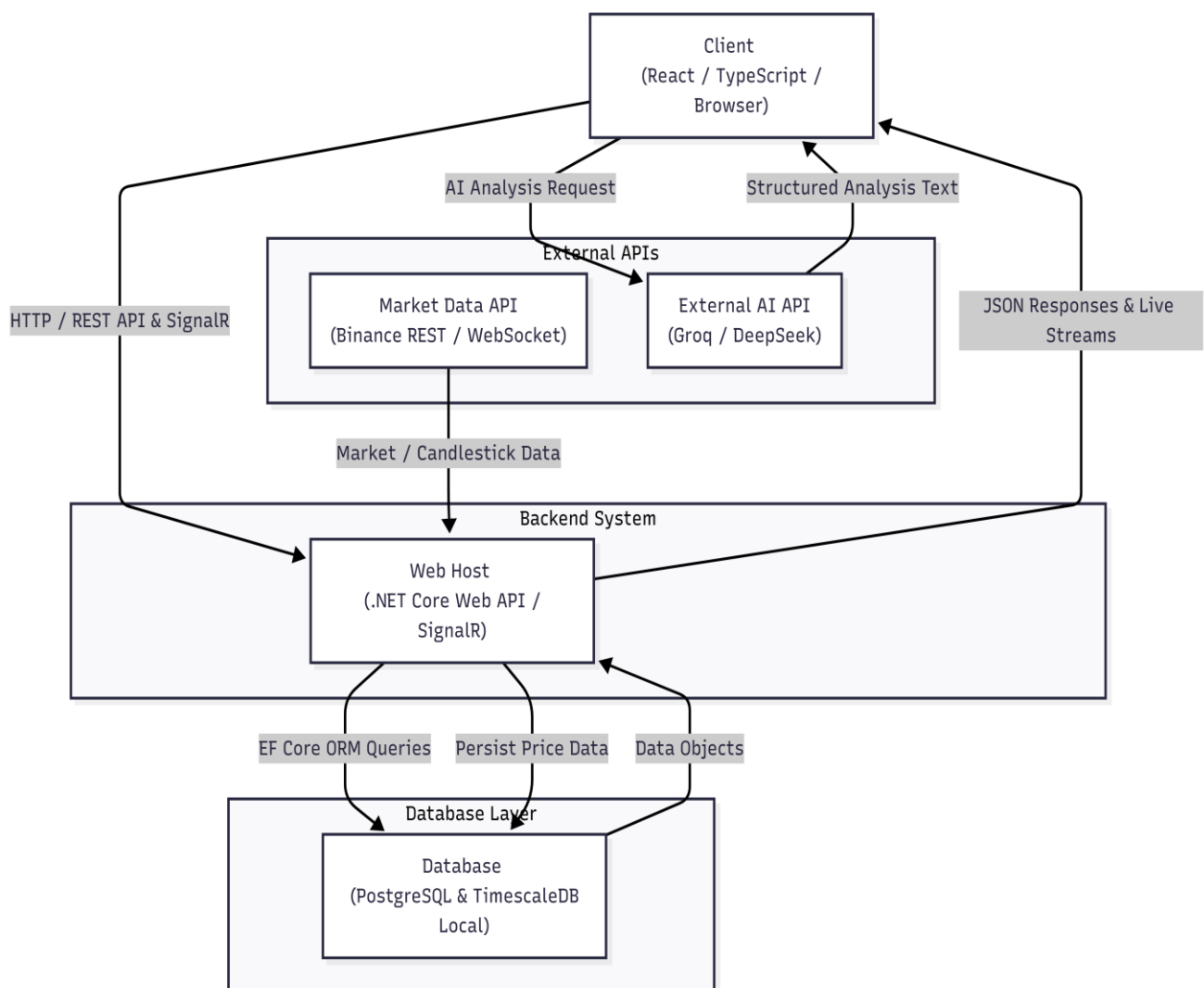
Webapplikationen udvikles med et responsivt design, så brugerfladen automatisk tilpasser sig forskellige skærmstørrelser og enheder. Dette sikrer, at systemets funktioner kan anvendes på både desktop, tablet og mobil.

Brugerfladen udvikles med fokus på enkel og intuitiv navigation, så brugeren hurtigt kan skifte mellem kryptovalutaer, tidsrammer og interaktive candlestick-grafer. AI-analyser integreres direkte i brugerfladen, så relevante analyser kan tilgås uden at forlade systemet.

5.2 Systemflow (Arkitekturdiagram)

Systemflowet viser, hvordan systemets forskellige komponenter kommunikerer med hinanden. Brugeren interagerer med frontend-applikationen, som kommunikerer med backendens API. Backend håndterer autentificering, forretningslogik og databasekommunikation samt modtager realtidsdata fra eksterne tjenester.

Arkitekturen består primært af **React/TypeScript** i frontend, **ASP.NET Core Web API** i backend samt **PostgreSQL og TimescaleDB** til datalagring. Realtime data modtages blandt andet via **WebSocket** og videreformidles til frontend gennem **SignalR**.



6. Tidsplan (Sprint-struktur over 4 uger)

Den nedenstående tidsplan er en overordnet plan for udviklingen af **TradingPro** over fire uger. Udviklingen gennemføres iterativt, hvor implementering, test og dokumentation udføres løbende gennem hele projektperioden.

DAGE	UGE			
	32	33	34	35
Mandag	Arbejde med case-beskrivelse	Opsætning af React og TypeScript frontend	Implementering af AI-analysetjenesten	Afsluttende systemtest og fejlrettelser
Tirsdag	Færdiggørelse af case-beskrivelse	Udvikling af dashboard og brugergrænseflade	Integration af AI-analyse i backend	Færdiggørelse og optimering af produktet
Onsdag	Aflevering af case-beskrivelse og påbegyndelse af kravspecifikation	Færdiggørelse og aflevering af kravspecifikation	Udvikling af AI-panelet	Rapportskrivning og dokumentation
Torsdag	Arbejde videre med kravspecifikation	Integration af TradingView Lightweight Charts	Implementering af User, Admin og SuperAdmin	Færdiggørelse af rapport og gennemgang
Fredag	Udvikling af projektstruktur og teknisk planlægning	Implementering af tidsrammer og SignalR	Samlet test og fejlrettelse	Endelig gennemgang og komplet aflevering

Bilag 1 – Use Cases

Brugsscenarie: Brugerregistrering og sikker login	Ident: UC-1
Aktør: Bruger / Administrator	
Beskrivelse: Brugeren opretter en konto eller logger ind for at få adgang til applikationens funktioner.	
Forudsætning: Aktøren har de nødvendige loginoplysninger eller kan oprette en konto.	
Forventet resultat: Brugeren bliver autentificeret og får adgang til sin profil og relevante funktioner.	
Forløb: 1. Aktøren åbner login- eller registreringssiden. 2. Aktøren indtaster de nødvendige oplysninger. 3. Systemet validerer oplysningerne. 4. Brugeren bliver logget ind og får adgang til applikationen.	
Alternativt forløb: 1. Aktøren indtaster ugyldige oplysninger. 2. Systemet viser en fejlmeddelelse. 3. Aktøren kan forsøge igen.	

Brugsscenarie: Realtids-markedsmonitorering	Ident: UC-2
Aktør: Bruger	
Beskrivelse: Brugeren vælger et handelssymbol og følger den aktuelle markedspris gennem en realtidsopdateret graf.	
Forudsætning: Brugeren er logget ind, og forbindelsen til Binance API fungerer.	
Forventet resultat: Brugeren kan se aktuelle prisdata, som opdateres løbende på grafen.	
Forløb: 1. Brugeren åbner markedsvisningen. 2. Brugeren vælger et symbol, f.eks. ETH/USDT. 3. Systemet henter aktuelle markedsdata fra Binance. 4. Data sendes til frontend via SignalR. 5. Grafen opdateres automatisk.	
Alternativt forløb: 1. Forbindelsen til Binance API afbrydes. 2. Systemet registrerer forbindelsesfejlen. 3. Brugeren informeres om, at realtidsdata ikke er tilgængelige.	

Brugsscenarie: AI-baseret markedsanalyse	Ident: UC-3
Aktør: Bruger	
Beskrivelse: Brugeren anmoder om en AI-genereret analyse af et valgt aktiv baseret på aktuelle og historiske markedsdata.	
Forudsætning: Brugeren er logget ind, og relevante markedsdata er tilgængelige.	
Forventet resultat: Brugeren modtager en teknisk og letforståelig analyse af det valgte aktiv.	
Forløb: 1. Brugeren vælger et aktiv. 2. Brugeren åbner AI-analysepanelet. 3. Brugeren anmoder om en analyse. 4. Systemet sender relevante data til AI-tjenesten. 5. AI-tjenesten genererer analysen. 6. Analysen vises i applikationen.	
Alternativt forløb: 1. AI-tjenesten kan ikke kontaktes. 2. Systemet viser en fejlmeddelelse. 3. Brugeren kan forsøge igen senere.	

Brugsscenarie: Historisk data-navigation	Ident: UC-4
Aktør: Bruger	
Beskrivelse: Brugeren navigerer gennem historiske candlestick-data og vælger forskellige tidsrammer for det valgte aktiv.	
Forudsætning: Brugeren har valgt et handelssymbol, og historiske data findes i TimescaleDB.	
Forventet resultat: Systemet viser de relevante historiske markedsdata i grafen.	
Forløb: 1. Brugeren vælger et handelssymbol. 2. Brugeren vælger en tidsramme, f.eks. 1 minut eller 1 time. 3. Systemet henter de relevante data fra TimescaleDB. 4. Data behandles og sendes til frontend. 5. Grafen opdateres med den valgte tidsramme.	
Alternativt forløb: 1. Der findes ingen data for den valgte tidsramme. 2. Systemet viser en besked om manglende data.	

Brugsscenarie: Brugerstyring og rolletildeling	Ident: UC-5
Aktør: Superadministrator	
Beskrivelse: Superadministratoren administrerer brugere og ændrer deres roller og adgangsniveauer.	
Forudsætning: Superadministratoren er logget ind og har de nødvendige rettigheder.	
Forventet resultat: Brugerens rolle og adgangsniveau bliver opdateret og gemt i systemet.	
Forløb: 1. Superadministratoren åbner brugeradministrationen. 2. Superadministratoren vælger en bruger. 3. Superadministratoren vælger en ny rolle, f.eks. Admin. 4. Systemet validerer ændringen. 5. Den nye rolle gemmes i PostgreSQL.	
Alternativt forløb: 1. Superadministratoren forsøger at ændre en ugyldig rolle. 2. Systemet afviser ændringen. 3. Systemet viser en fejlmeddelelse.	

Brugsscenarie: API-integration og systemovervågning	Ident: UC-6
Aktør: Administrator	
Beskrivelse: Administratoren overvåger forbindelsen mellem applikationen og Binance API for at sikre en stabil datastrøm.	
Forudsætning: Administratoren er logget ind, og systemet har adgang til Binance API.	
Forventet resultat: Administratoren kan se API-forbindelsens status og identificere eventuelle forbindelsesproblemer.	
Forløb: 1. Administratoren åbner systemovervågningen. 2. Systemet kontrollerer forbindelsen til Binance API. 3. Systemet viser forbindelsens aktuelle status. 4. Administratoren kan overvåge datastrømmen.	
Alternativt forløb: 1. Binance API svarer ikke. 2. Systemet registrerer forbindelsesfejlen. 3. Administratoren informeres om problemet.	